INVESTIGACIÓN DE SEGURIDAD WEB BÁSICA

proyecto Sistema de Gestión Clínica

investigación para documentación y seguimiento



1. Resumen ejecutivo

Este documento presenta una investigación introductoria sobre seguridad web y adapta sus recomendaciones al Sistema de Gestión Clínica. El proyecto contempla información de pacientes, historias clínicas, turnos, consultas, internaciones, medicamentos y datos administrativos. Por ello, la seguridad debe considerarse desde el diseño y durante todo el desarrollo.

Se toman como referencias OWASP Top 10:2025, las guías OWASP para prevenir inyección y el Secure Software Development Framework (SSDF) del NIST. Este documento no certifica que la aplicación sea segura ni reemplaza una auditoría o prueba de penetración.

Objetivo y alcance

Identificar riesgos web relevantes y definir medidas preventivas para las tareas de backend, base de datos, integración y pruebas. El contexto considerado incluye backend Java con Spring Boot, los portales de pacientes y personal, y las bases sistema_clinico y administracion_clinica. Las recomendaciones deben contrastarse con el código y el despliegue real antes de afirmar que están implementadas.

- Incluye amenazas comunes, autenticación, autorización, validación, protección de datos, configuración, auditoría y pruebas.
- No incluye explotación de sistemas reales, auditoría de código, certificación ni verificación de cumplimiento legal.

2. ¿Qué es la seguridad web?

La seguridad web reúne prácticas técnicas y organizativas para proteger aplicaciones, usuarios y datos frente a accesos no autorizados, modificaciones indebidas, filtraciones y caídas. Busca mantener la confidencialidad, integridad y disponibilidad de la información.

En una clínica, una falla puede exponer información personal o médica, permitir cambios indebidos en turnos o medicamentos, o comprometer datos administrativos.

3. Riesgos web relevantes

OWASP Top 10:2025 agrupa riesgos importantes de las aplicaciones web. Los siguientes son especialmente relevantes para el sistema.

Riesgo	Significado	Ejemplo en la clínica	Prevención
Pérdida de control de acceso	Un usuario accede a datos o funciones ajenos a sus permisos.	Un paciente ve la Historia Clínica de otra persona.	Verificar permisos en el servidor en cada operación; aplicar mínimo privilegio.
Configuración insegura	Servicios, permisos o parámetros quedan expuestos.	Errores muestran detalles internos o credenciales quedan en el repositorio.	Separar entornos, cerrar funciones innecesarias y proteger secretos.
Fallos criptográficos	Datos sensibles no se protegen adecuadamente.	Contraseñas en texto plano o tráfico sin HTTPS.	Hash adaptativo de contraseñas y HTTPS en despliegue.
Inyección	Entradas se interpretan como parte de una consulta o comando.	Un campo de búsqueda altera una consulta SQL.	Consultas parametrizadas, validación y permisos mínimos.
Diseño inseguro	Los flujos no contemplan amenazas.	El backend confía en que la interfaz oculte botones administrativos.	Validar reglas y permisos en el servidor.

Riesgo	Significado	Ejemplo en la clínica	Prevención
Pérdida de control de acceso	Un usuario accede a datos o funciones ajenos a sus permisos.	Un paciente ve la Historia Clínica de otra persona.	Verificar permisos en el servidor en cada operación; aplicar mínimo privilegio.
Fallos de autenticación	El inicio de sesión o las sesiones pueden ser abusados.	Intentos ilimitados o sesión válida después de cerrar sesión.	Límites de intentos, sesiones seguras y recuperación controlada.
Fallos de registro y monitoreo	No hay registros suficientes para detectar incidentes.	No se sabe quién consultó o modificó un registro clínico.	Registrar accesos y cambios relevantes sin guardar secretos ni datos innecesarios.

4. Medidas de seguridad recomendadas

4.1 Autenticación y contraseñas

- El DNI puede utilizarse como identificador si así lo requiere el proyecto, pero no es un secreto ni reemplaza la contraseña.
- Nunca almacenar contraseñas en texto plano ni mediante cifrado reversible. Usar hash adaptativo como Argon2id, bcrypt o una alternativa segura soportada.
- Limitar intentos de inicio de sesión y evitar mensajes que revelen si un DNI está registrado.
- Proteger la recuperación de contraseña mediante un proceso verificable, de un solo uso y con vencimiento.
- Invalidar sesiones al cerrar sesión; si se usan cookies, evaluar atributos Secure, HttpOnly y SameSite.

4.2 Autorización y roles

El proyecto contempla portales diferenciados y perfiles funcionales vinculados al personal, como médico, enfermero, farmacéutico, jefatura y administrador. La autorización debe comprobarse en el backend, no solo en la interfaz.

- Definir una matriz de permisos para consultar, crear, modificar y cancelar cada tipo de dato.
- Comprobar que los pacientes accedan solo a sus propios datos y que el personal acceda únicamente a lo necesario para su función.
- No confiar en identificadores enviados por el navegador sin comprobar que el usuario tenga permiso sobre ese recurso.
- Separar permisos de la aplicación y privilegios de las cuentas que acceden a cada base de datos.
- Aplicar mínimo privilegio y revisar los permisos cuando cambien las responsabilidades.

4.3 Validación de entradas, SQL Injection y XSS

Los datos recibidos desde formularios, parámetros o servicios externos deben considerarse no confiables. Validar en el backend aunque el frontend también valide.

- Validar tipo, longitud, formato y rango: por ejemplo, fechas, DNI, cantidades de medicamentos e identificadores.
- Usar consultas parametrizadas; no concatenar texto ingresado por el usuario dentro de sentencias SQL.
- Revisar los procedimientos almacenados: utilizarlos no elimina automáticamente el riesgo si construyen SQL dinámico de forma insegura.
- Codificar la salida según el contexto y evitar insertar contenido no confiable como HTML sin sanitización para reducir XSS.
- Si se permiten archivos adjuntos, validar tipo, tamaño, nombre y permisos de almacenamiento.

4.4 Protección de datos clínicos y administrativos

- Usar HTTPS para transmitir credenciales y datos sensibles en entornos desplegados.
- Limitar los datos personales y médicos mostrados en pantallas, respuestas API, reportes y errores.

- No guardar contraseñas, tokens, datos clínicos completos ni secretos en logs, archivos públicos o repositorios Git.
- Restringir y proteger las copias de seguridad de ambas bases de datos y probar su restauración.
- Controlar quién puede exportar, imprimir o descargar información clínica y registrar operaciones sensibles.
- Evaluar el cifrado de datos almacenados y backups según riesgos, infraestructura y obligaciones aplicables.

4.5 Backend Java / Spring Boot

- Centralizar autenticación y autorización mediante mecanismos adecuados de Spring Security.
- Proteger todos los endpoints, incluidos aquellos que no están enlazados desde la interfaz.
- Gestionar secretos mediante variables de entorno o un gestor de secretos; no subir credenciales al repositorio.
- Mantener Java, Spring Boot y dependencias actualizados y revisar vulnerabilidades conocidas.
- Configurar CORS de forma restrictiva cuando corresponda; CORS no reemplaza autenticación ni autorización.
- Si se usan cookies y sesiones, evaluar protección CSRF; si se usan tokens, definir expiración, revocación, almacenamiento y transporte seguros.
- Mostrar mensajes de error genéricos al usuario y guardar solo el detalle técnico necesario en registros protegidos.

4.6 Dos bases de datos y procedimientos almacenados

El proyecto distingue sistema_clinico, para la operatoria clínica, y administracion_clinica, para procesos como sueldos, recibos, pagos y facturación. Aunque estén conectadas, cada acceso debe limitarse a la necesidad funcional.

- Usar cuentas de base de datos con permisos mínimos, evitando usuarios administradores para la operación habitual.
- Revisar procedimientos clínicos como AgregarPaciente, BuscarHistoriaClinica, RegistrarMedicacion y CrearTurno, y administrativos como RegistrarSueldo, GenerarRecibo y CrearFactura.
- Validar parámetros y permisos antes de ejecutar procedimientos.
- Definir manejo de errores y consistencia para operaciones que involucren ambas bases.
- No devolver errores SQL detallados al usuario final.

4.7 Auditoría y notificaciones

- Registrar inicio de sesión exitoso o fallido, cambios de permisos, consultas o modificaciones sensibles de Historia Clínica, cambios de stock y operaciones administrativas relevantes.
- Incluir fecha/hora, usuario, acción, resultado e identificador afectado cuando corresponda, evitando duplicar datos médicos innecesarios.
- Proteger los logs frente a modificación no autorizada y limitar quién puede consultarlos.
- Si se utilizan notificaciones por Telegram, no enviar historias clínicas, contraseñas ni datos sensibles. Preferir avisos mínimos que remitan a un portal autenticado.
- Definir qué eventos generan alertas y quién debe revisarlas.

5. Recomendaciones priorizadas

Prioridad	Acción	Componente	Criterio de validación
Alta	Definir matriz de roles y permisos y aplicarla en backend.	API y portales.	Un usuario sin permiso recibe acceso denegado.
Alta	Usar hash seguro de contraseñas y proteger sesiones.	Login de pacientes y personal.	No hay contraseñas en texto plano; cerrar sesión invalida la sesión.

Alta	Usar consultas parametrizadas y revisar procedimientos.	Backend y ambas bases.	Entradas especiales no alteran consultas ni revelan datos.
Alta	Verificar autorización sobre cada paciente y recurso.	Historia Clínica, turnos, internaciones y medicación.	Cambiar un ID no permite consultar registros ajenos.
Alta	Proteger secretos, configuración y tráfico.	Spring Boot, despliegue y repositorio.	No hay credenciales en Git y HTTPS está habilitado en el entorno publicado.
Media	Auditar operaciones sensibles.	Clínica, stock y administración.	Las pruebas generan registros útiles sin exponer secretos.
Media	Revisar dependencias y errores.	Backend y compilación.	Se revisan dependencias vulnerables y no se muestran trazas internas.
Media	Proteger backups y probar restauración.	Ambas bases de datos.	Acceso restringido y restauración documentada.
Media	Agregar pruebas de seguridad negativas.	Pruebas e integración.	Se prueban accesos indebidos, entradas inválidas y sesiones.

6. Plan básico de validación

Las pruebas deben ejecutarse en un entorno de desarrollo o pruebas autorizado y con datos ficticios, no contra sistemas ajenos ni con historias clínicas reales innecesariamente expuestas.

1. Autenticación: probar contraseña incorrecta, usuario inexistente, intentos repetidos, cierre de sesión y expiración.
2. Autorización: probar cuentas de paciente, médico, enfermero, farmacéutico y administrador con permisos distintos.
3. Aislamiento de datos: intentar consultar un recurso de otro paciente cambiando el identificador; el servidor debe denegar el acceso.
4. Validación e inyección: probar caracteres especiales y valores fuera de rango en búsquedas, turnos y medicamentos.
5. Errores: provocar fallos controlados y comprobar que no se muestren trazas internas.
6. Auditoría: verificar que operaciones sensibles registren usuario, fecha, acción y resultado.
7. Configuración: revisar dependencias, secretos, permisos de bases de datos y configuración del entorno.

8. Conclusiones

La seguridad web del Sistema de Gestión Clínica depende especialmente del control de acceso, la protección de credenciales, la validación de datos y el cuidado de la información clínica y administrativa. Tener portales separados ayuda a organizar el sistema, pero no basta: el backend debe comprobar permisos en cada operación.

Esta investigación ofrece una base para convertir los riesgos identificados en tareas de backend, base de datos, integración y pruebas. Las decisiones finales deben revisarse frente a la arquitectura real y los requisitos legales aplicables al tratamiento de datos personales y de salud.

9. Fuentes consultadas

- OWASP Foundation. OWASP Top 10:2025 (español). https://top10.owasp.org/2025/es/
- OWASP Foundation. SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Foundation. Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.html
- NIST. Secure Software Development Framework (SSDF), SP 800-218. https://csrc.nist.gov/pubs/sp/800/218/final
- NIST. Traducción al español del SSDF 1.1. https://www.nist.gov/publications/secure-software-development-framework-ssdf-version-11-recommendations-mitigating-risk-0

Nota metodológica: las fuentes externas respaldan los riesgos y prácticas generales. La adaptación a módulos, roles, bases de datos y procedimientos se basa en el contexto conocido del proyecto y debe validarse contra la implementación vigente.